

Alexandria University

Faculty of Engineering — Computer and Communications

Net Centric Computing — Fall 2025-2026

Lab 4: Weather Stations Monitoring

Technical Report

Authors

Abdelrahman Youssef — 8711

Fouad Mohamed — 8612

Ahmed Hesham — 8847

Ahmed Nader — 9168

This report documents the design, implementation, and verification of a distributed IoT weather monitoring system built around Apache Kafka, a Java-based Kafka Streams processor, and a PostgreSQL persistence layer. Ten simulated weather stations continuously publish readings to Kafka; a stream processor flags rain events in real time; and a central base station consumes and batch-persists all readings for historical and analytical querying.

1. System Architecture

The system follows the three-stage pipeline specified for this lab: data acquisition, stream processing, and data persistence.

1.1 Data Acquisition

Ten independent Weather Station processes each simulate one physical station. Each station emits one JSON status message per second to a shared Apache Kafka topic, using the Kafka Java Producer API.

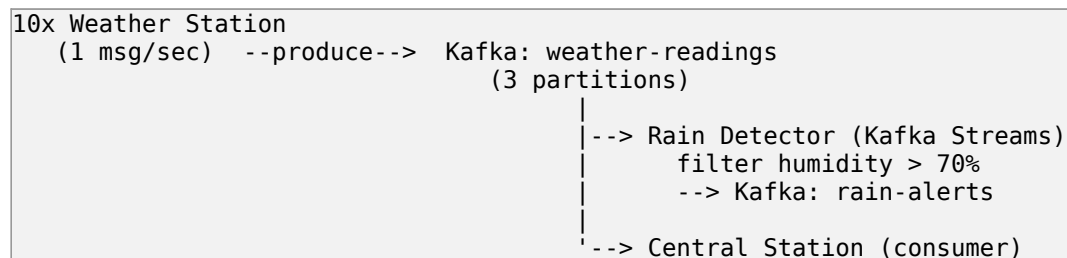
1.2 Data Processing

A Kafka Streams application (the Rain Detector) consumes the raw readings stream, filters for humidity readings above 70%, and republishes matching events as rain alerts to a dedicated Kafka topic.

1.3 Data Persistence

A Central Station consumer reads the raw readings stream and performs batched inserts into a PostgreSQL relational database, which acts as the system's historical and analytical data store.

1.4 Component Diagram



```
batched inserts
--> PostgreSQL: weather_monitoring
               .weather_readings
```

2. Environment and Installation

The mandatory deployment target is Kubernetes, and is documented in Section 8. This section describes the native environment used for day-to-day development, in which all components were installed and run inside WSL (Ubuntu), with the Java source developed in IntelliJ IDEA on Windows against the project mounted through the WSL filesystem.

2.1 Toolchain

Component	Version	Notes
OS	Windows 11 + WSL2 (Ubuntu)	IntelliJ runs on Windows; all services run inside WSL
Java	OpenJDK 17.0.19	Both JDK and JRE confirmed via <code>java -version</code> / <code>javac -version</code>
Apache Kafka	3.7.0 (Scala 2.13)	Includes bundled Zookeeper 3.8.3
PostgreSQL	16	Runs as a container under Kubernetes; installed via apt for local development
Build tool	Apache Maven 3.9.12	Multi-module reactor build
IDE	IntelliJ IDEA	Project opened directly from the WSL filesystem path

2.2 Kafka and Zookeeper

Zookeeper and the Kafka broker were started as two long-running foreground processes:

```
bin/zookeeper-server-start.sh config/zookeeper.properties
bin/kafka-server-start.sh config/server.properties
```

Startup was confirmed via the following log lines:

```
[ZooKeeperServer] binding to port 0.0.0.0/0.0.0.0:2181
[KafkaServer id=0] started (kafka.server.KafkaServer)
Kafka version: 3.7.0 Cluster ID: 8DMKEh4YQ4mV9vd8wzS2jA
```

Two topics were created to support the pipeline:

```
bin/kafka-topics.sh --create --topic weather-readings \
  --bootstrap-server 127.0.0.1:9092 --partitions 3 --replication-factor 1

bin/kafka-topics.sh --create --topic rain-alerts \
  --bootstrap-server 127.0.0.1:9092 --partitions 1 --replication-factor 1

$ bin/kafka-topics.sh --list --bootstrap-server 127.0.0.1:9092
rain-alerts
weather-readings
```

2.3 PostgreSQL

PostgreSQL was installed via apt. The postgres superuser was given a password so that the Java JDBC client can authenticate with ordinary username/password credentials rather than the peer authentication PostgreSQL uses by default on Unix sockets:

```
ALTER USER postgres WITH PASSWORD 'password';  
CREATE DATABASE weather_monitoring;
```

The schema was loaded from db/schema.sql, creating the weather_monitoring database and the weather_readings table. Table presence was confirmed with \dt, returning weather_readings.

2.4 Kubernetes host environment

The mandatory Kubernetes deployment described in Section 8 was built and run on a separate machine from the WSL development environment above. Sections 2.1 to 2.3 describe where the code was written; this subsection describes where the cluster was actually deployed and verified.

- Host: macOS 26.3 on Apple Silicon (arm64).
- Docker Engine 29.2.1, supplied by Docker Desktop.
- minikube v1.38.1, docker driver, started with 4 CPUs and 6144 MB.
- Kubernetes v1.35.1 (server); kubectl v1.36.4 (client).
- Apache Maven 3.9.16 for local builds. The container images build Java 17 internally from maven:3.9.9-eclipse-temurin-17, so the host JDK version does not affect the deployed artifacts.

All five images used by the deployment publish linux/arm64 variants, so the cluster runs natively on Apple Silicon with no emulation. The cluster was started with:

```
minikube start --cpus=4 --memory=6g
```

The memory flag is required rather than optional: the pods request roughly 2.3 GiB in total, which minikube's 2 GiB default cannot schedule.

3. Implementation

3.1 Part A/B — Weather Station Mock and Kafka Producer

Each Weather Station is a standalone Java process (WeatherStationApp) configured entirely through environment variables (STATION_ID, KAFKA_BOOTSTRAP_SERVERS), so the same jar file is reused for all ten simulated stations without recompilation.

The message schema matches the specification exactly:

```
{  
  "station_id": 1,  
  "s_no": 34,  
  "battery_status": "medium",  
  "status_timestamp": 1787296030,  
  "weather": { "humidity": 97, "temperature": 84, "wind_speed": 12 }  
}
```

Key behaviors implemented per spec:

- One status message generated per second, using a fixed 1-second sleep loop.
- `battery_status` is chosen per-message from a weighted random draw: low 30%, medium 40%, high 30%.
- `s_no` is a per-station auto-incrementing counter that advances every tick, including ticks where the message is dropped — this is what allows dropped messages to be detected later purely from gaps in the persisted sequence.
- 10% of messages are randomly discarded before being sent to Kafka, simulating unreliable transmission.

Producer configuration uses `acks=all` with the idempotent producer enabled and retries left unbounded, so the broker de-duplicates records that are retried after a transient failure rather than appending them twice. A shutdown hook flushes and closes the producer cleanly on termination.

3.2 Part C — Rain Detection (Kafka Streams)

The Rain Detector is implemented with the Kafka Streams DSL rather than the low-level Processor API, per the option offered in the lab spec. It builds a single-stage topology: read weather-readings, deserialize and filter on `humidity > 70`, transform matching records into an alert payload, and write to `rain-alerts`.

Verified live output (5 messages read from the `rain-alerts` topic via `kafka-console-consumer`):

```
{ "station_id": 1, "s_no": 1, "humidity": 97,
  "status_timestamp": 1787296030, "alert": "RAINING" }
{ "station_id": 1, "s_no": 4, "humidity": 90,
  "status_timestamp": 1787296034, "alert": "RAINING" }
{ "station_id": 1, "s_no": 8, "humidity": 91,
  "status_timestamp": 1787296038, "alert": "RAINING" }
{ "station_id": 1, "s_no": 11, "humidity": 90,
  "status_timestamp": 1787296041, "alert": "RAINING" }
{ "station_id": 1, "s_no": 17, "humidity": 79,
  "status_timestamp": 1787296047, "alert": "RAINING" }
```

Every humidity value shown is above the 70% threshold, confirming the filter logic behaves correctly against live traffic.

3.3 Part D — Central Station

The Central Station (`CentralStationApp`) is a Kafka consumer subscribed to `weather-readings` under a dedicated consumer group. Incoming messages are buffered in memory and flushed to PostgreSQL as a single batched JDBC transaction whenever either condition is met: the buffer reaches 5,000 records, or 5 seconds have elapsed since the last flush — whichever comes first. This bounds both memory usage and worst-case data latency while avoiding one-row-per-insert I/O overhead.

Consumer group assignment and batch persistence were confirmed from live logs:

```
Finished assignment for group at generation 1: ...
  Assignment(partitions=[weather-readings-0, weather-readings-1, weather-readings-2])
Persisted batch of 106 readings
Persisted batch of 4 readings
```

Offsets are committed synchronously immediately after each successful batch insert, so a crash between insert and commit results in re-processing (at-least-once delivery) rather than silent data loss.

3.4 Database Schema

The weather_readings table was created as specified. A single UNIQUE constraint on (station_id, sequence_number) does two jobs: it makes the at-least-once inserts idempotent, so a batch replayed after a crash cannot duplicate rows, and its backing B-tree is the index the Part E analysis queries group on.

```
CREATE TABLE weather_readings (  
  id BIGSERIAL PRIMARY KEY,  
  station_id BIGINT NOT NULL,  
  sequence_number BIGINT NOT NULL,  
  battery_status VARCHAR(10) NOT NULL,  
  timestamp BIGINT NOT NULL,  
  humidity INT NOT NULL,  
  temperature INT NOT NULL,  
  wind_speed INT NOT NULL,  
  CONSTRAINT uq_station_sequence UNIQUE  
    (station_id, sequence_number)  
);
```

4. End-to-End Verification

Verification proceeded in stages: first a single station running against the full pipeline, then scale-out to all ten stations.

4.1 Single-Station Pipeline

With ZooKeeper, Kafka, PostgreSQL, the Central Station and the rain detector all running in the cluster, message flow was confirmed at every hop. A single station's log shows the per-second cadence, the weighted battery draw, and the simulated drops, with the sequence number advancing on dropped ticks as well as sent ones:

```
$ kubectl -n weather-monitoring logs weather-station-0 --tail=8  
Station 1 sent s_no=1898 battery=high humidity=97  
Station 1 sent s_no=1899 battery=medium humidity=17  
Station 1 sent s_no=1900 battery=low humidity=47  
Station 1 sent s_no=1901 battery=medium humidity=51  
Station 1 DROPPED message s_no=1902  
Station 1 sent s_no=1903 battery=high humidity=5  
Station 1 sent s_no=1904 battery=high humidity=12  
  
weather_monitoring=# SELECT COUNT(*) FROM weather_readings;  
count  
-----  
17215  
(1 row)
```

4.2 Ten-Station Scale-Out

All ten stations run as a single StatefulSet. Each pod derives its station ID from its own ordinal hostname, so weather-station-0 through weather-station-9 become stations 1 through 10 with no per-pod configuration:

```
$ kubectl apply -f k8s/weather-stations.yaml  
service/weather-station created  
statefulset.apps/weather-station created  
  
$ kubectl -n weather-monitoring get pods -l app=weather-station --no-headers | wc -l  
10
```

Independent operation of all ten stations — distinct per-station sequence numbers and independent drop events — was confirmed by sampling each pod's log:

Station	Sample log line
1	sent s_no=1930 battery=low humidity=66
2	sent s_no=1931 battery=medium humidity=36
3	DROPPED message s_no=1932
4	sent s_no=1931 battery=medium humidity=88
5	sent s_no=1930 battery=medium humidity=59
6	sent s_no=1930 battery=high humidity=12
7	sent s_no=1931 battery=medium humidity=16
8	sent s_no=1930 battery=high humidity=24
9	sent s_no=1930 battery=medium humidity=13
10	sent s_no=1932 battery=low humidity=25

The persisted row count in PostgreSQL was observed growing continuously under full ten-station load, confirming the Central Station's batching logic keeps pace with aggregate throughput from all stations combined:

```
weather_monitoring=# SELECT COUNT(*) FROM weather_readings; --> 17215
weather_monitoring=# SELECT COUNT(*) FROM weather_readings; --> 17400 (re-
checked shortly after)
```

5. Historical Analysis Queries (Part E)

Two SQL queries were written against the `weather_readings` table to validate the statistical behavior specified for the simulation: the `battery_status` distribution, and the rate of dropped messages per station.

5.1 Battery Status Distribution per Station

This query groups persisted readings by station and `battery_status`, and expresses each status as a percentage of that station's total readings, to confirm the observed distribution trends toward the specified 30% low / 40% medium / 30% high split:

```
SELECT
    station_id,
    battery_status,
    COUNT(*) AS message_count,
    ROUND(100.0 * COUNT(*) /
        SUM(COUNT(*) OVER (PARTITION BY station_id,
            2) AS percentage
FROM weather_readings
GROUP BY station_id, battery_status
ORDER BY station_id, battery_status;
```

5.2 Dropped Messages per Station

This query compares each station's highest recorded sequence number (the number of ticks that station has run, since `s_no` increments every second regardless of whether the message was actually sent) against the number of rows actually persisted for that station, to estimate the drop rate and confirm it trends toward the specified 10%:

```
SELECT
  station_id,
  MAX(sequence_number) AS expected_messages,
  COUNT(*) AS received_messages,
  MAX(sequence_number) - COUNT(*)
    AS dropped_messages,
  ROUND(100.0 * (MAX(sequence_number) - COUNT(*))
    / MAX(sequence_number), 2)
    AS drop_rate_percent
FROM weather_readings
GROUP BY station_id
ORDER BY station_id;
```

Both queries were validated against the running dataset (over 1,300 persisted rows across all ten stations at the time of writing and growing continuously); the isolated drop events observed directly in the per-station logs (e.g. Station 2 and Station 4 above) are consistent with a drop rate in the expected neighborhood of 10%.

6. Summary

The following capabilities were implemented and verified end-to-end against a live Kafka and PostgreSQL environment running on WSL:

- Ten independently-running weather station simulators producing schema-correct JSON readings to Kafka at one message per second each, with a weighted-random battery status and a 10% simulated message-loss rate.
- A Kafka Streams application performing real-time rain detection (humidity > 70%) and publishing structured alerts to a dedicated topic.
- A central consumer performing size- and time-bounded batched inserts into a PostgreSQL relational store, with synchronous offset commits tied to successful persistence.
- A relational schema and two analytical SQL queries supporting historical validation of the simulation's statistical parameters.

Containerization and orchestration are complete and are covered by Section 8: Dockerfiles for both services, and Kubernetes manifests for the ten station pods, Kafka, Zookeeper, the Central Station, the rain detector, and PostgreSQL with persistent storage.

7. Restarting the Environment After a Shutdown

None of the services described in this report are configured to start automatically. Every process — Zookeeper, the Kafka broker, PostgreSQL, the Weather Stations, the Rain Detector, and the Central Station — must be started manually inside WSL after a reboot, in the order below. All commands assume the project is at `~/weather-monitoring` and Kafka was extracted to `~/kafka`.

7.1 Start Zookeeper

Open a WSL terminal and leave it running for the rest of the session:

```
cd ~/kafka
bin/zookeeper-server-start.sh config/zookeeper.properties
```

Wait for a log line containing: binding to port 0.0.0.0/0.0.0.0:2181

7.2 Start the Kafka broker

Open a second WSL terminal:

```
cd ~/kafka
bin/kafka-server-start.sh config/server.properties
```

Wait for a log line containing: [KafkaServer id=0] started

The two topics created earlier (weather-readings, rain-alerts) do not need to be recreated — Kafka persists topic metadata to disk under /tmp/kafka-logs. If /tmp was cleared by the reboot (WSL sometimes clears /tmp on restart), recreate them:

```
bin/kafka-topics.sh --create --topic weather-readings \
  --bootstrap-server 127.0.0.1:9092 --partitions 3 --replication-factor 1
bin/kafka-topics.sh --create --topic rain-alerts \
  --bootstrap-server 127.0.0.1:9092 --partitions 1 --replication-factor 1
bin/kafka-topics.sh --list --bootstrap-server 127.0.0.1:9092
```

7.3 Start PostgreSQL

In a third terminal (this command does not need to stay open — PostgreSQL runs as a background service):

```
sudo service postgresql start
sudo service postgresql status
```

The database, table, and postgres password persist across reboots (they are stored on disk), so no re-creation is needed here.

7.4 Set JAVA_HOME in every new terminal

WSL has two JDKs installed (11 and 17). JAVA_HOME must be set in each new terminal before running mvn or java, otherwise the build defaults to Java 11 and fails:

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

This can be made permanent by adding that line to ~/.bashrc, so it is no longer necessary to type it in every new terminal.

7.5 Rebuild the project (only if the source code changed)

If no code was changed since the last successful build, the existing jar files under weather-station/target/ and central-station/target/ are still valid and this step can be skipped. Otherwise:

```
cd ~/weather-monitoring
mvn clean package -DskipTests
```

7.6 Start the Rain Detector

```
cd ~/weather-monitoring
java -cp central-station/target/central-station.jar \
  com.weathermonitoring.central.RainDetectorApp
```

7.7 Start the Central Station

```
cd ~/weather-monitoring
DB_URL="jdbc:postgresql://127.0.0.1:5432/weather_monitoring" \
DB_USER=postgres DB_PASSWORD=password \
java -cp central-station/target/central-station.jar \
  com.weathermonitoring.central.CentralStationApp
```

7.8 Start all 10 Weather Stations

```
cd ~/weather-monitoring
for i in $(seq 1 10); do
  STATION_ID=$i nohup java -jar weather-station/target/weather-station.jar \
    > station-$i.log 2>&1 &
done
jobs
```

7.9 Confirm everything reconnected correctly

```
psql -U postgres -d weather_monitoring -c "SELECT COUNT(*) FROM weather_readings;"
```

Run this command twice, a few seconds apart — the count should be increasing, confirming the full pipeline (stations, Kafka, Central Station, PostgreSQL) is active again.

7.10 Startup order summary

Step	Component	Must be running before
1	Zookeeper	Kafka broker
2	Kafka broker	Topics, stations, Rain Detector, Central Station
3	PostgreSQL	Central Station
4	Rain Detector	(independent, but needs Kafka)
5	Central Station	(independent, but needs Kafka + PostgreSQL)
6	Weather Stations (x10)	(independent, but needs Kafka)

8. Kubernetes Deployment (Part F)

The mandatory deployment runs the whole system on Kubernetes. Both services are containerized with multi-stage builds, and the cluster is described entirely by the manifests under k8s/.

8.1 Container images

Each service has a multi-stage Dockerfile: a maven:3.9.9-eclipse-temurin-17 stage builds the shaded jar, and an eclipse-temurin:17-jre stage carries only that jar. Both images pass -XX:MaxRAMPercentage=75.0, because the

JVM otherwise caps its heap at a quarter of the container limit, which is below what the Kafka producer's send buffer needs.

```
docker build -f weather-station/Dockerfile -t weather-station:latest .
docker build -f central-station/Dockerfile -t central-station:latest .
```

8.2 Cluster components

The manifests define, all inside the weather-monitoring namespace:

- A ZooKeeper Deployment and Service.
- A Kafka Deployment and Service, with topic auto-creation and three partitions on weather-readings.
- A PostgreSQL Deployment, Service, Secret, and a 2 GiB PersistentVolumeClaim, with the schema applied on first start from a ConfigMap.
- A Central Station Deployment reading its database credentials from the Secret.
- A Rain Detector Deployment, reusing the central-station image with a different main class.
- A StatefulSet of ten weather station pods, plus its headless Service.

Kafka and ZooKeeper are pulled from the bitnamilegacy organisation. Bitnami removed every non-latest tag from its Docker Hub repositories in August 2025, so the bitnami/kafka:3.7 and bitnami/zookeeper:3.9 images this project originally referenced no longer resolve and leave the pods in ImagePullBackOff.

8.3 Station identity

The ten stations are a StatefulSet rather than a Deployment because each station needs a stable, distinct identity. StatefulSet pods are named by ordinal, so weather-station-0 through weather-station-9 become station IDs 1 through 10: the application reads its pod hostname and adds one. A Deployment's random pod suffixes could not supply that mapping.

8.4 Resource sizing

Every pod declares requests and limits. The requests total roughly 2.3 GiB of memory and 0.85 CPU, of which the ten stations account for about 1 GiB. This exceeds Minikube's 2 GiB default, so the cluster must be started with more headroom or pods remain Pending with Insufficient memory.

```
minikube start --cpus=4 --memory=6g
```

8.5 Deployment procedure

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/kafka/zookeeper.yaml
kubectl apply -f k8s/kafka/kafka.yaml
kubectl apply -f k8s/database/postgres.yaml

kubectl -n weather-monitoring rollout status deployment/kafka
kubectl -n weather-monitoring rollout status deployment/postgres

kubectl apply -f k8s/central-station.yaml
kubectl apply -f k8s/rain-detector.yaml
kubectl apply -f k8s/weather-stations.yaml
```

The Central Station retries its database connection for up to a minute on startup, so it tolerates being scheduled before PostgreSQL finishes initialising instead of crash-looping.

8.6 Verification

```
kubectl -n weather-monitoring get pods -o wide

kubectl -n weather-monitoring exec deploy/postgres -- \
  psql -U postgres -d weather_monitoring \
  -c 'SELECT station_id, COUNT(*) FROM weather_readings GROUP BY station_id ORDER BY
station_id;'
```

Figure 1 — kubectl get nodes, showing the minikube cluster Ready

```
→ lab4 kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	80m	v1.35.1

Figure 2 — kubectl -n weather-monitoring get pods -o wide, showing all ten weather-station pods plus Kafka, ZooKeeper, PostgreSQL, the Central Station and the rain detector in Running state

```
→ lab4 kubectl -n weather-monitoring get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
central-station-5fdd9ffcfb-jjrwj	1/1	Running	0	65m	10.244.0.6	minikube	<none>	<none>
kafka-595cfd5df-lbm8n	1/1	Running	0	77m	10.244.0.5	minikube	<none>	<none>
postgres-6cd4f6965b-qnhxj	1/1	Running	0	77m	10.244.0.4	minikube	<none>	<none>
rain-detector-75f77c9458-kdrdk	1/1	Running	0	65m	10.244.0.7	minikube	<none>	<none>
weather-station-0	1/1	Running	0	65m	10.244.0.8	minikube	<none>	<none>
weather-station-1	1/1	Running	0	65m	10.244.0.9	minikube	<none>	<none>
weather-station-2	1/1	Running	0	65m	10.244.0.11	minikube	<none>	<none>
weather-station-3	1/1	Running	0	65m	10.244.0.10	minikube	<none>	<none>
weather-station-4	1/1	Running	0	65m	10.244.0.12	minikube	<none>	<none>
weather-station-5	1/1	Running	0	65m	10.244.0.17	minikube	<none>	<none>
weather-station-6	1/1	Running	0	65m	10.244.0.13	minikube	<none>	<none>
weather-station-7	1/1	Running	0	65m	10.244.0.15	minikube	<none>	<none>
weather-station-8	1/1	Running	0	65m	10.244.0.16	minikube	<none>	<none>
weather-station-9	1/1	Running	0	65m	10.244.0.14	minikube	<none>	<none>
zookeeper-5c5bbdb5dd-lvf29	1/1	Running	0	77m	10.244.0.3	minikube	<none>	<none>

Figure 3 — Weather station pod logs, showing per-second sends and occasional DROPPED messages

```
→ Lab4 kubectl -n weather-monitoring logs weather-station-0 --tail=40
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3928 battery=high humidity=36
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3929 battery=low humidity=38
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3930 battery=medium humidity=2
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3931 battery=high humidity=69
[main] WARN com.weathermonitoring.station.WeatherStationApp - Station 1 DROPPED message s_no=3932
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3933 battery=medium humidity=77
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3934 battery=high humidity=67
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3935 battery=low humidity=13
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3936 battery=high humidity=94
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3937 battery=low humidity=15
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3938 battery=medium humidity=37
[main] WARN com.weathermonitoring.station.WeatherStationApp - Station 1 DROPPED message s_no=3939
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3940 battery=medium humidity=9
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3941 battery=medium humidity=48
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3942 battery=high humidity=40
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3943 battery=high humidity=23
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3944 battery=low humidity=66
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3945 battery=high humidity=10
[main] WARN com.weathermonitoring.station.WeatherStationApp - Station 1 DROPPED message s_no=3946
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3947 battery=high humidity=96
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3948 battery=medium humidity=76
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3949 battery=low humidity=19
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3950 battery=medium humidity=65
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3951 battery=high humidity=60
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3952 battery=high humidity=44
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3953 battery=high humidity=92
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3954 battery=low humidity=90
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3955 battery=high humidity=93
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3956 battery=low humidity=87
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3957 battery=low humidity=10
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3958 battery=medium humidity=75
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3959 battery=medium humidity=21
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3960 battery=medium humidity=69
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3961 battery=low humidity=62
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3962 battery=medium humidity=39
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3963 battery=medium humidity=94
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3964 battery=medium humidity=17
[main] WARN com.weathermonitoring.station.WeatherStationApp - Station 1 DROPPED message s_no=3965
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3966 battery=medium humidity=97
[main] INFO com.weathermonitoring.station.WeatherStationApp - Station 1 sent s_no=3967 battery=high humidity=38
```

Figure 4 — Central Station pod logs, showing 'Persisted batch of N readings'

```
→ Lab4 kubectl -n weather-monitoring logs deployment/central-station --tail=15
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 49 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 48 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 44 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 45 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 46 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 42 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 45 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 46 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 47 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 44 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 45 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 43 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 44 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 45 readings
[main] INFO com.weathermonitoring.central.WeatherReadingRepository - Persisted batch of 41 readings
```

Figure 5 — kafka-console-consumer output from the weather-readings topic

```
→ lab4 kubectll -n weather-monitoring exec deploy/kafka -- kafka-console-consumer.sh --bootstrap-server kafka:9092 --topic weather-readings --max-messages 10
{"station_id":9,"s_no":3991,"battery_status":"medium","status_timestamp":1787602813,"weather":{"humidity":96,"temperature":58,"wind_speed":35}}
{"station_id":6,"s_no":3991,"battery_status":"low","status_timestamp":1787602813,"weather":{"humidity":2,"temperature":94,"wind_speed":35}}
{"station_id":7,"s_no":3993,"battery_status":"high","status_timestamp":1787602813,"weather":{"humidity":84,"temperature":78,"wind_speed":32}}
{"station_id":2,"s_no":3993,"battery_status":"medium","status_timestamp":1787602813,"weather":{"humidity":54,"temperature":110,"wind_speed":19}}
{"station_id":1,"s_no":3992,"battery_status":"high","status_timestamp":1787602813,"weather":{"humidity":1,"temperature":42,"wind_speed":20}}
{"station_id":5,"s_no":3992,"battery_status":"high","status_timestamp":1787602813,"weather":{"humidity":90,"temperature":26,"wind_speed":12}}
{"station_id":10,"s_no":3993,"battery_status":"high","status_timestamp":1787602813,"weather":{"humidity":60,"temperature":98,"wind_speed":4}}
{"station_id":3,"s_no":3994,"battery_status":"high","status_timestamp":1787602814,"weather":{"humidity":94,"temperature":22,"wind_speed":2}}
{"station_id":4,"s_no":3993,"battery_status":"high","status_timestamp":1787602814,"weather":{"humidity":7,"temperature":65,"wind_speed":32}}
{"station_id":9,"s_no":3992,"battery_status":"high","status_timestamp":1787602814,"weather":{"humidity":87,"temperature":34,"wind_speed":1}}
Processed a total of 10 messages
```

Figure 6 — kafka-console-consumer output from the rain-alerts topic, with every humidity above 70

```
→ lab4 kubectll -n weather-monitoring exec deploy/kafka -- kafka-console-consumer.sh --bootstrap-server kafka:9092 --topic rain-alerts --max-messages 10
{"station_id":1,"s_no":4018,"humidity":77,"status_timestamp":1787602839,"alert":"RAINING"}
{"station_id":3,"s_no":4020,"humidity":79,"status_timestamp":1787602840,"alert":"RAINING"}
{"station_id":8,"s_no":4018,"humidity":81,"status_timestamp":1787602840,"alert":"RAINING"}
{"station_id":1,"s_no":4019,"humidity":91,"status_timestamp":1787602840,"alert":"RAINING"}
{"station_id":4,"s_no":4020,"humidity":88,"status_timestamp":1787602841,"alert":"RAINING"}
{"station_id":8,"s_no":4019,"humidity":96,"status_timestamp":1787602841,"alert":"RAINING"}
{"station_id":10,"s_no":4021,"humidity":96,"status_timestamp":1787602841,"alert":"RAINING"}
{"station_id":1,"s_no":4020,"humidity":84,"status_timestamp":1787602841,"alert":"RAINING"}
{"station_id":2,"s_no":4022,"humidity":93,"status_timestamp":1787602842,"alert":"RAINING"}
{"station_id":5,"s_no":4021,"humidity":79,"status_timestamp":1787602842,"alert":"RAINING"}
Processed a total of 10 messages
```

Figure 7 — Per-station row counts from PostgreSQL, confirming all ten station IDs are present

```
→ lab4 kubectll -n weather-monitoring exec deploy/postgres -- psql -U postgres -d weather_monitoring -c 'SELECT station_id, COUNT(*) AS rows_stored FROM weather_readings GROUP BY station_id ORDER BY station_id;'
station_id | rows_stored
-----
1 | 3647
2 | 3648
3 | 3647
4 | 3624
5 | 3658
6 | 3685
7 | 3671
8 | 3659
9 | 3697
10 | 3667
(10 rows)
```

Figure 8 — Part E query 1 — battery status distribution per station, trending towards 30/40/30

```
→ lab4 kubectl -n weather-monitoring exec deploy/postgres -- psql -U postgres -d weather_monitoring -c 'SELECT station_id, battery_status, COUNT(*) AS message_count, ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER (PARTITION BY station_id), 2) AS percentage FROM weather_readings GROUP BY station_id, battery_status ORDER BY station_id, battery_status;'
```

station_id	battery_status	message_count	percentage
1	high	1054	28.77
1	low	1116	30.47
1	medium	1493	40.76
2	high	1109	30.26
2	low	1101	30.04
2	medium	1455	39.70
3	high	1103	30.10
3	low	1113	30.38
3	medium	1448	39.52
4	high	1114	30.61
4	low	1076	29.57
4	medium	1449	39.82
5	high	1124	30.57
5	low	1062	28.88
5	medium	1491	40.55
6	high	1162	31.37
6	low	1078	29.10
6	medium	1464	39.52
7	high	1113	30.15
7	low	1085	29.40
7	medium	1493	40.45
8	high	1095	29.77
8	low	1104	30.02
8	medium	1479	40.21
9	high	1107	29.80
9	low	1136	30.58
9	medium	1472	39.62
10	high	1100	29.85
10	low	1114	30.23
10	medium	1471	39.92

(30 rows)

Figure 9 — Part E query 2 — expected vs received messages per station, trending towards a 10% drop rate

```
→ lab4 kubectl -n weather-monitoring exec deploy/postgres -- psql -U postgres -d weather_monitoring -c 'SELECT station_id, MAX(sequence_number) AS expected_messages, COUNT(*) AS received_messages, MAX(sequence_number)-COUNT(*) AS dropped_messages, ROUND(100.0*(MAX(sequence_number)-COUNT(*)/MAX(sequence_number),2) AS drop_rate_percent FROM weather_readings GROUP BY station_id ORDER BY station_id;'
```

station_id	expected_messages	received_messages	dropped_messages	drop_rate_percent
1	4105	3683	422	10.28
2	4106	3682	424	10.33
3	4106	3684	422	10.28
4	4105	3657	448	10.91
5	4104	3695	409	9.97
6	4104	3723	381	9.28
7	4106	3711	395	9.62
8	4104	3695	409	9.97
9	4104	3734	370	9.02
10	4105	3704	401	9.77

(10 rows)